

AGABI Backend Phase 2 — Master Implementation Blueprint

Agabi Engineering

July 22, 2026

CONTENTS

AGABI Backend Phase 2 — Master Implementation Blueprint	
Global rules for every milestone	
Preconditions (must be true before M0)	
M0 · Knowledge spine (schema + store)	
M1 · Ingestion pipeline (pure stages)	
M2 · Extractor (advisor) + validation gates	
M3 · Review (CLI) + first verified chapter	
M4 · Search + path selection	
M5 · Teaching bridge ⚠ the first student-visible milestone	
M6 · Review UI + trust automation	
M7 · Teaching Knowledge Layer ⚠ the milestone that tests the thesis	
M8 · Observation store	
M9 · Assessment + breadth + releases	
Sequence and gates	
Effort	
Standing rules	

AGABI Backend Phase 2 — Master Implementation Blueprint

This is not architecture. Architecture is frozen in the Final Baseline. This is execution.

Every milestone is independently shippable, leaves the build green, and requires no architectural decision. Where a decision appears, it is a **deferred decision (§34)** with a stated default — take the default and record the observation.

Global rules for every milestone

- `npx tsc --noEmit`, `npm run lint`, `npx vitest run`, `npm run build` all clean before a milestone is complete. Stop `next dev` before `build`.
- **STOP AND ASK before any** `prisma db push`. Tanveer runs it himself.
- **STOP AND ASK before any new dependency.** M0–M9 need none.
- Every milestone ends in one commit with a message stating what invariant it establishes.
- No milestone may weaken a test from an earlier milestone.

Preconditions (must be true before M0)

#	Precondition	Status
P1	Evidence spine landed; reconstruction proof passes	in flight
P2	<code>prisma db push</code> run for <code>Workspace.title/subject</code>	BLOCKING — never run
P3	Multi-canvas verified at runtime in a browser	blocked on P2

M0 does not start until P1–P3 are green. Grounding without measurement means no way to know whether it helped.

M0 · Knowledge spine (schema + store)

Objective. The knowledge tables exist, the store interface exists, and the conformance suite passes against Postgres and an in-memory implementation.

Why. Everything else writes through this. Getting the interface wrong is the one mistake that propagates into every later milestone.

Dependencies. P1–P3.

Inputs. Final Baseline §10 (schema), §9 (module layout), §17 (storage).

Files created.

```
src/server/knowledge/ids.ts
src/server/knowledge/concept.ts
src/server/knowledge/statement.ts
src/server/knowledge/context/{registry,canonical,match}.ts
src/server/knowledge/graph/{dependency,composition,reinforcement,traverse}.ts
src/server/knowledge/store/KnowledgeStore.ts
src/server/knowledge/store/postgres.ts
src/server/knowledge/store/memory.ts
src/server/knowledge/store/conformance.test.ts
```

Database changes. All L1–L3 tables from §10: `Source`, `SourceChunk`, `Provenance`, `Concept`, `ConceptAlias`, `ConceptTag`, `DependencyEdge`, `CompositionEdge`, `ReinforcementEdge`, `ContextDimension`, `Context`, `Statement`, `Contradiction`, `ReviewEvent`. **One db push, gated.**

APIs. None public. `KnowledgeStore` interface only.

Tests.

- `conformance.test.ts` — run against both implementations
- `dependency-dag` — whole-graph acyclicity
- `composition-dag`
- `reinforcement-cycles-pass` — **asserts a deliberate cycle does NOT fail**
- `graph-conflict` — no pair in dependency and reinforcement, same direction
- `context-canonical` — identical dimension sets hash identically; key order irrelevant
- `identity` — no FK targets `slug`; `Concept.id` never updated

- `no-delete` — no destructive path in `knowledge/`
- `empty-platform` — every method returns sensible empty results on zero rows
- `walls` — extend `architecture.test.ts` with W1–W7

Rollback. Drop the new tables. Nothing else references them.

Migration. Purely additive.

Risks. Store interface wrong → every later milestone leaks Prisma. Mitigated by writing the memory implementation *first* and forcing the interface to be storage-agnostic.

Acceptance. Both store implementations pass one conformance suite. A cycle inserted into `ReinforcementEdge` passes CI; the same cycle in `DependencyEdge` fails with the cycle path printed.

Complete when. All tests green, build clean, committed.

M1 · Ingestion pipeline (pure stages)

Objective. A PDF becomes deterministic, content-addressed, locator-preserving chunks.

Why. Determinism here is what makes extraction re-runnable when a better model appears.

Dependencies. None — pure functions, testable without M0.

Files.

```
src/server/ingest/parse/{pdf,markdown,html}.ts
src/server/ingest/{clean,normalise,chunk,pipeline}.ts
src/server/ingest/spans.ts           // Span[] – offsets survive every transform
```

Database. None.

Tests.

- determinism: ingest the same bytes twice, byte-compare all chunk ids
- span preservation: after clean + normalise, a known quote's char range still maps to the correct page
- cleaning does not over-strip (fixture with headers, footers, exercise numbering)
- re-ingestion diff: change one paragraph, assert exactly one chunk id changes

Rollback. Delete the directory.

Risks. Locator loss is unrecoverable (§12.2). Mitigated by span-based representation and the span-preservation test.

Acceptance. One real NCERT chapter produces stable chunks; a one-paragraph edit produces exactly one new chunk id.

M2 · Extractor (advisor) + validation gates

Objective. A chunk produces validated `MACHINE_PROPOSED` / `AUTO_VALIDATED` proposals. Nothing reaches the graph.

Why. This is where the trust boundary is enforced. Extraction is the only model call in Phase 2.

Dependencies. M0, M1.

Files.

```
src/server/advisors/knowledge/extractEntities.ts
src/server/advisors/knowledge/extractStatements.ts
src/server/advisors/knowledge/extractDependencies.ts
src/server/advisors/knowledge/{prompts,schemas}.ts // PROMPT_VERSION lives here
src/server/knowledge/trust/validators/{grounding,payload,units,dates,refs,scope}.ts
src/server/knowledge/trust/{ladder,policy}.ts
```

Database. None beyond M0.

Tests.

- `grounding` — a fabricated quote is rejected; **use the RFC-1 Appendix A “photosynthesis releases nitrogen” case as a fixture**
- V1–V15 each with a passing and a failing fixture
- `trust-gate` — nothing above `AUTO_VALIDATED` without a human `ReviewEvent`
- `walls` — the extractor’s return type has no trust field (compile-level assertion)
- golden-set scoring harness (scores recorded, not yet gating)

Rollback. Disable the pipeline entry point; validators are pure and harmless.

Risks. Extractor quality unknown (D1). Take the four-pass default; record precision/recall against the golden set.

Acceptance. One chapter produces proposals; every one carries a quote literally present in its chunk; a hand-injected hallucination is rejected with zero human involvement.

M3 · Review (CLI) + first verified chapter

Objective. A human promotes proposals to `OFFICIAL_SOURCE_VERIFIED`. One chapter is genuinely verified.

Why. This is the first milestone that produces real knowledge, and the first measurement of review throughput.

Dependencies. M2.

Files.

```
src/server/review/{queue,batch,decide}.ts
src/server/knowledge/{review,version}.ts
scripts/review-cli.mjs           // batch view: source pane + proposals +
highlights
```

Database. `Release`, `ReleaseMember`.

Tests.

- `trust-gate` — promotion requires a `ReviewEvent` with `actorId`
- `provenance` — every promoted statement has provenance
- batch atomicity — a mid-batch failure commits nothing
- `no-silent-uncertainty` — reads below `labelBelow` are labelled

Rollback. Promotions are `ReviewEvent`s; demote by adding events. Nothing is lost.

Risks. Throughput below ~150/hr means the tooling is wrong and M6 must come earlier.

Measure and report.

Acceptance. ~150 concepts and ~400 statements at `OFFICIAL_SOURCE_VERIFIED` for NCERT Class 10 Science, Life Processes. **Report the measured statements-per-hour.**

Complete when. The chapter is verified and the golden set for it is hand-authored.

M4 · Search + path selection

Objective. `resolve("photosynthesis")` returns concepts; `selectPath` returns a prerequisite-ordered plan.

Dependencies. M3 (needs real content).

Files. `src/server/knowledge/{search,path,curriculum}.ts`

Database. `Program`, `ProgramNode`, `Mapping`, `LearningObjective`, `ObjectiveConcept`, `ClosureCache`.

Tests.

- runs 1–2 deterministic against fixtures
- `trustPolicy` is required — a call without it fails to compile
- `curriculum-independence` — drop all program rows, `selectPath` still works
- path determinism — same graph and seeds produce identical order (snapshot)
- closure cache invalidation on edge write

Rollback. Unused by teaching until M5.

Acceptance. `selectPath` returns prerequisites before core, topologically ordered, budget-bounded, identical across runs.

M5 · Teaching bridge the first student-visible milestone

Objective. A student typing “photosynthesis” receives a lesson where every block traces to a concept id.

Why. This is the milestone that proves the phase. Everything before it is scaffolding.

Dependencies. M4.

Files modified. `src/server/conversation/manager.ts` (one call site, §8.2) ·
`src/server/conversation/prompt.ts` (grounded prompt, bump `PROMPT_VERSION`) ·
`src/server/knowledge/path.ts` (`outlineFrom`).

Database. None.

Tests.

- grounded end-to-end: every block carries a concept id
- fallback: unknown topic → `defaultOutline` + `knowledge.miss` emitted
- `repairOutline` invariants still hold on grounded outlines — heading bookend, three-visual floor, max text run
- evidence: `grounded`, concept ids, versions, release recorded per lesson

Rollback. One line — revert `startLesson` to `defaultOutline(topic)`. This must remain true through M8.

Risks. R12 — grounding may not measurably help. That is the point of measuring.

Acceptance. Two lessons, one grounded and one not, indistinguishable in shape and distinguishable in evidence. **Report the quality comparison.**

M6 · Review UI + trust automation

Objective. 100 statements reviewed in under an hour. Statements reach `AUTO_VALIDATED` with no human involvement.

Dependencies. M5.

Files.

```
src/app/api/knowledge/review/{queue,batch,decide}/route.ts
src/app/knowledge/review/page.tsx           // batch screen, source pane, high-
lights
src/server/knowledge/trust/{corroboration,contradiction,inference,promote}.ts
src/server/review/{merge,split}.ts
```

Database. None beyond M0.

Tests.

- `trust-demotion` — a raised contradiction demotes within one cycle
- corroboration counts *publishers*, not documents
- contradiction blocks promotion above `AUTO_VALIDATED`
- merge: loser id resolves forever; no reference breaks
- **split: observations apportioned by recorded context; source resolves as AMBIGUOUS**
- `split-resolvable` — every reference carries usage context

Rollback. UI routes removable; CLI review remains.

Risks. R2 — ladder gamed by volume. Halt conditions plus publisher-level independence.

Acceptance. Measured review rate $\geq 150/\text{hr}$. A statement promotes to `AUTO_VALIDATED` untouched by a human, and a contradiction demotes it automatically.

M7 · Teaching Knowledge Layer ⚠️ the milestone that tests the thesis

Objective. Lessons pre-empt misconceptions and use verified analogies.

Why. §33.1 predicts this, not M5, is where quality moves.

Dependencies. M6.

Files.

```
src/server/knowledge/teaching/{asset,registry,select}.ts
src/server/knowledge/teaching/kinds/{misconception,analogy,workedExample}.ts
src/server/advisors/knowledge/extractAssets.ts
```

Database. `TeachingAsset`, `AssetEfficacy`.

Tests.

- `analogy-breakdown` — an `ANALOGY` without a breakdown point is **rejected**
- `assetsFor` respects trust policy and context
- `teaching.miss` emitted when no asset exists
- lesson composition includes a misconception pre-empt when one is available

Rollback. `assetsFor` returns empty → M5 behaviour.

Content. Author `MISCONCEPTION`, `ANALOGY`, `WORKED_EXAMPLE` for the verified chapter only.

Acceptance. A lesson names the “plants eat sunlight” misconception before introducing the equation. **Report quality versus M5.** If flat, §33.1 is falsified — stop and reconsider before M9.

M8 · Observation store

Objective. Observations recorded in a separate database; mastery computable as a query.

Dependencies. M7.

Files. `src/server/observation/{record,mastery,efficacy,earn}.ts`, `prisma/observation.prisma`.

Database. Separate instance. `Observation` only.

Tests.

- `store-separation` — no query joins the two stores
- mastery is a pure function of observations (no stored score)
- erasure: deleting a learner's observations leaves knowledge untouched
- `earn.ts` derives a `REINFORCES` edge from measured transfer

Rollback. Stop recording; knowledge unaffected.

Acceptance. A `REINFORCES` edge exists with `earned: true` that nobody authored.

M9 · Assessment + breadth + releases

Objective. Assessment items exist; every subject has one verified chapter; lessons replay exactly.

Dependencies. M8.

Files. `src/server/knowledge/assessment/{item, registry}.ts`,
`src/server/advisors/knowledge/extractItems.ts`.

Database. `AssessmentItem`, `ItemConcept`.

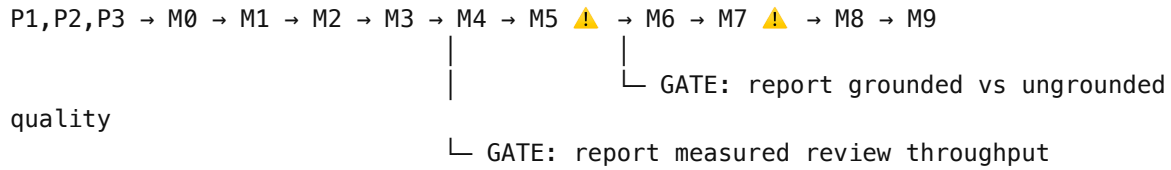
Content. Remaining Science → Maths (exercises `PROCEDURE`) → Social Science → English (exercises `SKILL`). **This is the 40–80 hour milestone.**

Tests.

- distractors carry `diagnosesMisconception`
- point-in-time replay: a lesson from three months ago resolves to the exact statement and asset versions
- all three knowledge kinds represented and teachable

Acceptance. Every subject has one fully verified chapter — proving the model holds across fact, procedure and skill.

Sequence and gates



Two hard gates. After M3, report review throughput — below 150/hr, do M6 before M5. After M7, report quality versus M5 — if both are flat, the thesis is falsified and M9’s content cost must not be incurred.

Effort

Milestone	Engineering	Content
M0–M2	3–4 weeks	–
M3	1 week	4–6 hrs
M4–M5	2 weeks	–
M6	2–3 weeks	–
M7	2 weeks	6–10 hrs
M8	1 week	–
M9	2 weeks	40–80 hrs

~13–15 weeks engineering. Content review dominates the schedule and is the real risk.

Standing rules

1. Never weaken an earlier milestone’s test.
2. Never add a field that stores a conclusion.
3. Never promote above `AUTO_VALIDATED` without a human.
4. Never unify the three graphs.
5. Never join the two stores.
6. Never serve `Provenance.quote` to a learner.

7. When a deferred decision surfaces, take the §34 default and record the observation.